CauseRuDi: Explaining Behavior Sequence Models by Causal Statistics Generation and Rule Distillation

Yao Zhang, Yun Xiong, Yiheng Sun, Yucheng Jin, Caihua Shan, Tian Lu, Hui Song, and Shengli Sun

***Abstract*—Risk scoring systems have been widely deployed in many applications, which assign risk scores to users according to their behavior sequences. Though many deep learning methods with sophisticated designs have achieved promising results, the black-box nature hinders their applications due to fairness, explainability, and compliance consideration. Rule-based systems are considered reliable in these sensitive scenarios. However, building a rule system is labor-intensive. Experts need to find informative statistics from user behavior sequences, design rules based on statistics and assign weights to each rule. In this paper, we bridge the gap between effective but black-box models and transparent rule models. We propose a two-stage framework, CauseRuDi, that distills the knowledge of black-box teacher models into rule-based student models. We design a Monte Carlo tree search-based statistics generation method that maximizes the correlation or dependence between the generated statistics and the teacher model's outputs. We formulate a sequential move game and a simultaneous move coalitional game to generate multiple statistics. Then statistics are composed into logical rules with our proposed neural logical networks by mimicking the outputs of teacher models. We evaluate CauseRuDi on three real-world public datasets and an industrial dataset to demonstrate its effectiveness.**

***Index Terms*—Rule distillation, sequence model explanation, statistics generation.**

I. Introduction

EXPONENTIAL growth in online activities has resulted in urgent needs for anomalous accounts detection to ensure the security and reliability of cyberspace. To evaluate risk scores of users, researchers and data scientists have proposed several approaches to model user behavior sequences, such as recurrent neural networks [1], or transformers [2].

Received 22 April 2023; revised 16 September 2024; accepted 17 October 2024. Date of publication 29 October 2024; date of current version 26 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62206059 and Grant U1936213, in part by China Postdoctoral Science Foundation under Grant 2022M710747, and in part by the Chinese National Key Laboratory of Science and Technology on Information System Security . Recommended for acceptance by Z. Guan. *(Corresponding author: Shengli Sun.)*

Yao Zhang, Yun Xiong, and Yucheng Jin are with the Shanghai Key Laboratory of Data Science, School of Computer Science, Fudan University, Shanghai 200437, China (e-mail: yaozhang@fudan.edu.cn; yunx@fudan.edu.cn; ycjin22@m.fudan.edu.cn).

Yiheng Sun and Hui Song are with the Tencent Weixin Group, Shenzhen 518054, China (e-mail: elisun@tencent.com; ivansong@tencent.com).

Caihua Shan is with the Microsoft Research Asia, Shanghai 200241, China (e-mail: caihuashan@microsoft.com).

Tian Lu is with the Department of Information Systems, Arizona State University, Tempe, AZ 85004 USA (e-mail: lutian@asu.edu).

Shengli Sun is with the School of Software and Microelectronics, Peking University, Beijing 100871, China (e-mail: slsun@pku.edu.cn).

Digital Object Identifier 10.1109/TKDE.2024.3487625

Though these advanced methods have achieved promising results, the nature of opacity hinders its broader application. For example, when users of an electronic payment platform complain when their accounts have been banned, customer services may need to dig into the raw transactions and figure out if they are real fraudsters. In conservative businesses, such as banks, the results of these black-box machine learning models are hardly used directly but needed to be reviewed by human experts first, as shown in Fig. 1(a). Instead, rule-based systems are considered more reliable in these scenarios, as shown in Fig. 1(b). A set of (weighted) rules are manually designed by human experts with their understanding and observations from historical data. Then if a user's behavior sequence meets the rules more than a pre-defined threshold, his/her account is banned without human intervention.

Though these rules are transparent and interpretable, they are limited by experts' experiences since experts may miss some patterns. Such systems are also hard to detect new risky patterns as fraudsters evolve. On the other hand, the rules are built upon a set of statistical features, e.g., *the number of unique devices*, concluded by experts. It is hard to measure whether these statistics are redundant, or whether they are the causal factors, i.e., direct causes or effects, of the risky behaviors [3].

To address the limitations of handcrafted rules, the researchers have proposed learning-based rule models [4], [5], [6], [7]. For example, the state-of-the-art method RRL [5] utilizes neural logical networks to learn a set of CNFs (conjunctive normal forms) or DNFs (disjunctive normal forms) from data. But these models require inputs being fixed-length feature vectors and cannot handle sequences of user behaviors. Moreover, due to the discrete nature of these models, they require stronger supervision signals to train.

In this paper, we bridge the gap between effective but opaque machine learning models and transparent but simple rule systems when modeling user behavior sequences. Instead of learning a logical rule model from raw data directly, we distill a trained risk scoring model, i.e., the teacher model, into a rule-based student model, as shown in Fig. 1(c). In this case, the precise predictions can be made by the teacher model, while the experts can audit the model by examining the distilled rules. Further, the distilled rules can supplement the existing rule systems to capture new trends and provide new insights.

Despite its significance, distilling rules from a sequence model is challenging due to the following reasons:

Exponential number of statistics. As Fig. 1(b) illustrates, rules are typically based on some statistics that human experts

1041-4347 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

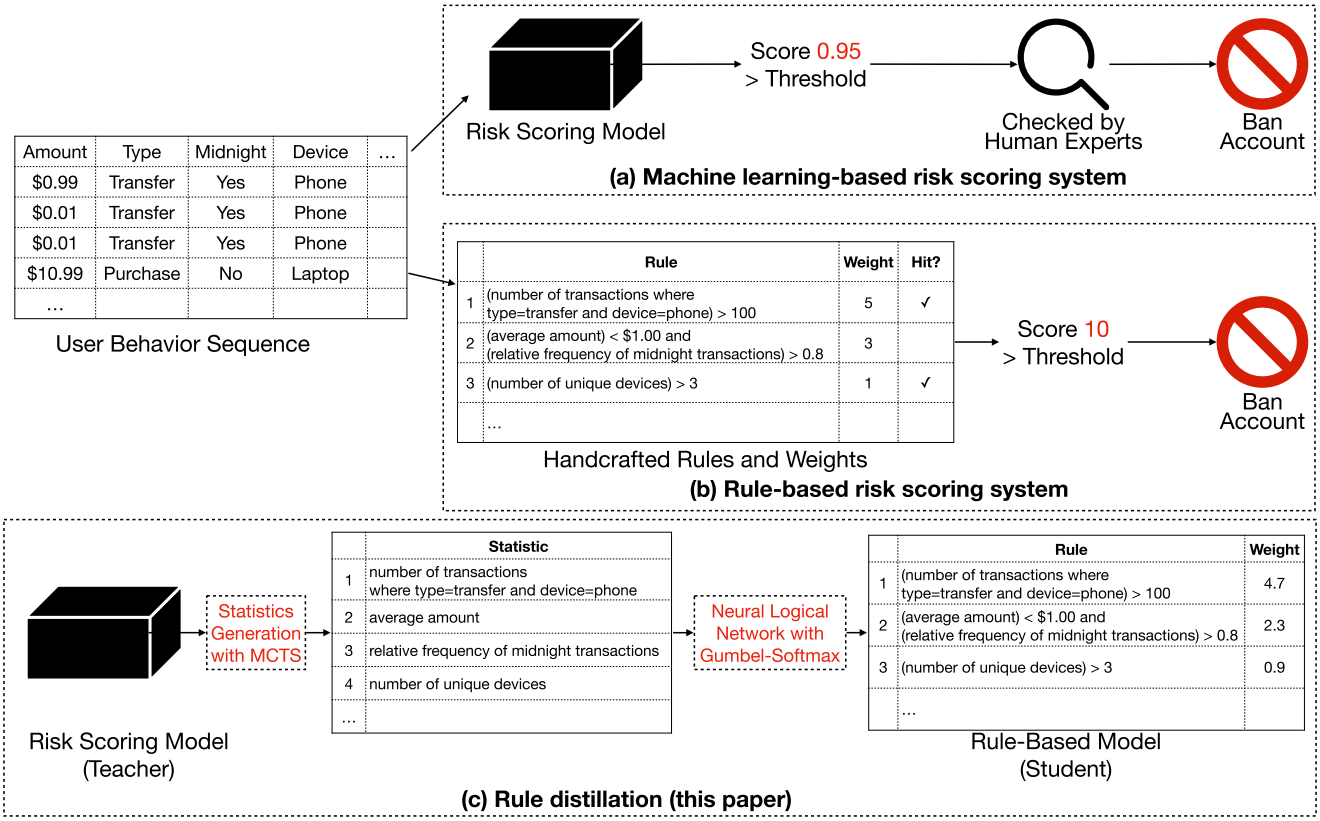


Fig. 1. Illustration of two types of risk scoring systems and rule distillation. (a) A black-box risk scoring model, despite its high precision, is hardly used without human review due to fairness, explainability, and compliance requirements. (b) Rule-based systems are considered more reliable in conservative businesses. However, designing rules are labor-intensive. (c) In this paper, we bridge the gap by proposing a two-stage rule distillation method CauseRuDi that can distill knowledge of black-box models into transparent rule-based systems.

handcraft. A set of informative statistics is the cornerstone of effective rules. However, the number of possible statistics grows exponentially as the number of conditions grows, e.g., the "where" clause in Fig. 1(b). Such a large space prevents us from enumerating all statistics, and a systematic search strategy must be applied.

Causality-based statistic selection. Even if we might find a systematic way to search the statistic space, it is prone to finding multiple redundant statistics, for example, the *average amount* and the *median amount*, which then result in redundant rules. Moreover, due to the nonlinearity of the teacher models, prevalent correlation-based feature (i.e., statistic) selection methods [3], [8] may fail. We have to explore the causal relationships between statistics and the outputs of the teacher models and test if they are mutually dependent. It is a challenge to select causal statistics while exploring the search space.

Tradeoff between effectiveness and interpretability of rule models. Unlike traditional knowledge distillation [9] where teachers and students share similar structures, rule distillation aims to use an interpretable rule model to mimic the outputs of a black-box teacher model. As pointed out by [5], traditional rule models like decision trees and rule lists/sets are hard to get comparable performance with complex models. Tree ensembles [10] though effective, are commonly not viewed as interpretable models. On the other hand, recently proposed neural logical networks [4], [5] produce complicated rules where each clause may contain over dozens of literals. These models are also hard to train due to their discrete structures. So it is a challenge to propose an effective and interpretable rule model that can precisely mimic the outputs of teacher models.

To tackle the above challenges, we propose an effective two-stage framework, named CauseRuDi (**Cau**sal **s**tatistics **G**eneration and **Ru**le **Di**stillation). In the first stage, we propose MCTS-SG, a novel **s**tatistics **g**eneration method based on **M**onte **C**arlo **T**ree **S**earch (MCTS) [11]. We pre-define a set of basic operators like Select, Mean, GroupBy, etc.. Then we model the statistics generation as a sequential decision process, where each statistic is composed of a series of operators. Since we need to generate multiple statistics, we formulate two different games. (1) We view the generation of multiple statistics as a sequential move game [11] where we generate statistics one after another. The objective of each move is to maximize the association between the current statistic and the teacher model's outputs, conditioning on the previously generated statistics. (2) We can also view the generation process as a simultaneous move coalitional game [12] where we grow multiple search trees simultaneously. The objective is to maximize the joint association between the generated statistics and the teacher model's outputs. To distribute the reward, we compute Shapley values [13] to measure the marginal contribution of each statistic. Each simulation

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

of tree search only involves a small batch of data, and the search space of statistics is explored by MCTS systematically. In this way, we generate a set of statistics effectively and efficiently.

In the second stage, we learn a set of weighted rules with our proposed GS-NLN. GS-NLN is a novel and lightweight **n**eural **l**ogical **n**etwork based on **G**umbel-**s**oftmax [14]. Specifically, each neuron in the GS-NLN only selects two neurons from the previous layer and puts them into a conjunctive or disjunctive formula. Unlike previous methods using logical activation functions [15] as surrogates, our proposed model directly uses boolean operations with Gumbel-softmax estimators and thus is easier to train. Thanks to the simplicity of structures, the learned rules are more interpretable and practicable than RRL. The experimental results prove that these simple rules can achieve satisfying performance.

The main contributions of this paper are as follows:

- We propose CauseRuDi, a two-stage framework that distills black-box behavior sequence models into rule models. The distilled rules are interpretable and transparent, and reveal the underlying mechanisms of teacher models. To the best of our knowledge, this is the first study on post hoc explanations of user behavior sequence models using rule distillation.
- We model the statistic generation as a sequential decision process of composing operators and design a Monte Carlo Tree Search-based search algorithm MCTS-SG. To generate multiple statistics, we formulate the sequential move game and the simultaneous move coalitional game. Our approach can generate causal statistics efficiently while only requiring small batches of data during tree searching.
- We propose a novel and lightweight neural logical network GS-NLN. Each neuron in the proposed model only connects to two neurons in the previous layer resulting in simpler forms of rules and better interpretability.
- Extensive experiments on three real-world datasets prove the effectiveness of our proposed CauseRuDi. Case studies on an industrial dataset demonstrate the interpretability of learned rules.

II. Related Work

In this section, we introduce three related topics: feature generation/selection, ruled-based models, and knowledge distillation.

Automatic Feature Generation and Feature Selection. Feature engineering is one of the most crucial and time-consuming steps in data mining tasks. For tabular data, many researchers [16], [17], [18], [19] have proposed automatic feature generation methods by doing feature combination and crossing. In [20] the authors proposed DSM, which enumerates features in relational databases. OneBM [21] extends DSM by introducing entity graph traversal methods and selecting features with Chi-squared tests. However, these methods cannot handle sequence data, not to mention extracting statistics from data. They also overlook the combination explosion problems and can only generate shallow features.

Another branch of related work is feature selection, which aims to select a subset of features from a large number of features for building downstream models. Early studies utilize correlation-based measures to select relevant features with respect to a target variable of interest [8]. However, correlations fail to capture the causal relationships between features and the target variable. Recent studies have shown that features that are with causal relationships with the target variable are more robust [22] and can improve the interpretability of downstream models [23]. Causality-based feature selection methods [3] instead select a subset of features that are direct causes or direct effects of the target variable. To measure causal relationships, (conditional) independence tests [24], [25] are largely adopted. There are some studies [26], [27] on feature engineering with Monte Carlo tree search (MCTS) [11], but they focus on feature selection for tabular data.

In this paper, we propose an MCTS-based statistics generation method MCTS-SG. We use the term "statistic" instead of "feature" because we treat it as a composition of basic operators. We systematically explore the search space of statistics with MCTS, and only small batches of data are used during searching. Different from feature selection, we do not explicitly enumerate all possible statistics and then select a subset of them. By defining basic operators, our proposed MCTS-SG can generate informative statistics effectively and efficiently.

Rule-based Models and Neural Logical Networks. It has been shown [5] that traditional rule models [4], [6], [7] with decision trees, rule lists, or rule sets are hard to get comparable performance. Moreover, most methods produce deep and hierarchical IF-THEN rules that are less practical in rule-based systems. In [5], the authors proposed the state-of-the-art rule-based model, RRL, which is a kind of neural network with logical activation functions [15]. RRL is hard to train due to its discrete structures. Besides, an extra continuous version of the network must be maintained to provide gradients. Though RRL learns flat rules, the number of literals in a rule is uncontrollable even with shallow network structures, leading to perplexing results.

In this paper, we propose GS-NLN, a Gumbel-softmax-based neural logical network with the constraint that each neuron can only connect to two neurons in the previous layer. This constraint simplifies the network structure and makes the model easy to train with Gumbel-softmax estimators [14]. The learned rules are also neat and sound.

Knowledge Distillation and Post-hoc Explanation with Surrogate Models. In [28] the authors proposed knowledge distillation, where a small student model is trained to mimic the outputs of a large teacher model. Following the idea, some researchers distill the knowledge of black-box models into transparent models [29], [30], i.e., surrogate models, as a way of post-hoc model explanation. For example, the model auditing method [29] detects what features were used in the teacher models. These approaches only produce importance of features. In [31], [32] the authors proposed methods to distill neural networks into decision trees. Another branch of research [13], [33] locally fits teacher models to generate instance-wise explanations. There is another work called rule knowledge distillation [34]. It distills rules into deep neural networks, which is opposite to our setting. Nonetheless, most existing methods cannot deal with sequence data.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

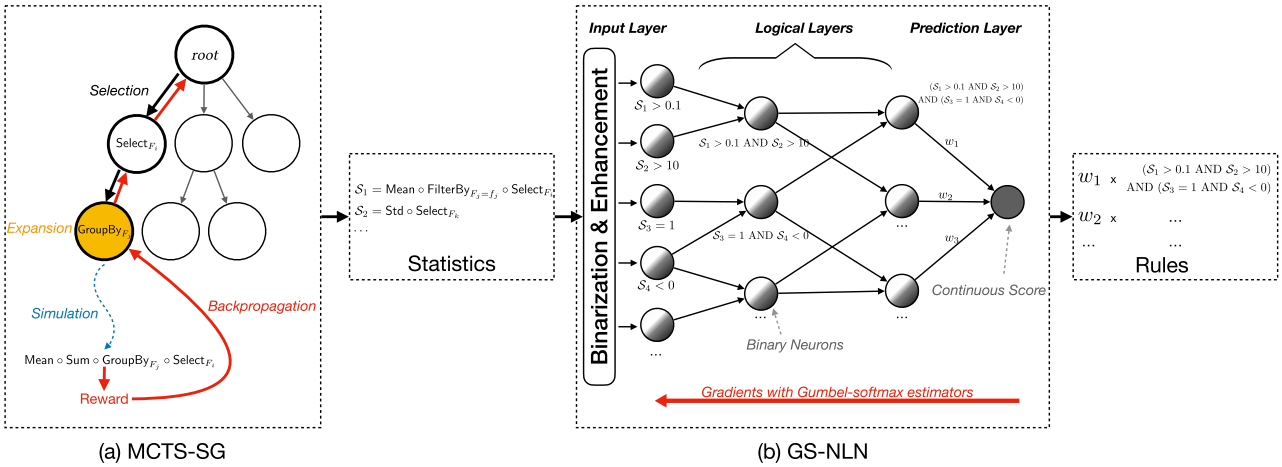


Fig. 2. The framework of our proposed CauseRuDi. (a) Statistics generation with Monte Carlo tree search. (b) Neural logical network with Gumbel-softmax. Here we discard disjunction parts and skip connections for simplicity.

In this paper, we distill a black-box sequence model into a set of rules with the help of the proposed MCTS-SG and GS-NLN. This paper is the first study on distilling sequence models into rule systems to the best of our knowledge.

III. Methodology

A. Overview

Let $\{\mathcal{X}_1, \ldots, \mathcal{X}_N\}$ denote the dataset of N users, where $\mathcal{X}_i$ is the sequence of user i's behaviors. All sequences contains m_1 categorical variables (columns) $\{F_1, \ldots, F_{m_1}\}$ and m_2 numerical variables (columns) $\{F_{m_1+1}, \ldots, F_{m_1+m_2}\}$. The sample spaces of categorical variables are denoted as $\mathbb{F}_j = \{f_{j1}, \ldots, f_{jp}\}$, while the sample spaces of the numerical ones are $\Re$. For notation simplicity, here we assume $|\mathbb{F}_1| = \cdots = |\mathbb{F}_{m_1}| = p$.

A trained risk scoring model, e.g., a recurrent neural network, takes the sequence $\mathcal{X}_i$ as input and produces its continuous prediction $y_i \in \Re$. Note that we do not constrain the range of y_i, since we treat the risk scoring model as a black-box, and its outputs may correspond to probabilities, log-probabilities, or just real-valued risk scores. The task of rule distillation is to find a set of weighted rules that can mimic the teacher model's outputs $y \in \Re^N$.

The framework of our proposed CauseRuDi is illustrated in Fig. 2. CauseRuDi is a two-stage method. In the first stage, we use an MCTS-based statistics generation method to generate a small set of statistics. Then in the second stage, we binarize the statistics into literals and train a neural logical network by minimizing the ranking loss with respect to y. After training, the weights and neurons in the last prediction layer are extracted as the distilled rules.

B. Stage 1: Statistics Generation With MCTS

In the first stage, we search for multiple promising statistics that are with causal relationships with the teacher model's outputs y and thus can be transformed into literals. We propose two different games, and the corresponding algorithms are summarized in Algorithm 1 and Algorithm 2.

1) Statistics as Compositions of Operators: In this paper, we define a statistic[1] as a composition of basic operators:

$$\mathcal{S} = \mathcal{T}_d \circ \cdots \circ \mathcal{T}_1,\ 2 \le d \le D,$$

where D is the maximum depth of the statistics, and $\mathcal{T}_i \in \mathbb{T}$ is selected from the following pre-defined sets:

$$\mathbb{T} = \mathbb{T}_{\text{select}} \cup \mathbb{T}_{\text{agg}} \cup \mathbb{T}_{\text{by}} \cup \mathbb{T}_{\text{tsfm}}.$$

If the readers are familiar to the *pandas*,[2] one of the most popular Python data analysis libraries, each statistic is analogous to a chain of pandas' operators. For example, $\mathsf{Mean} \circ \mathsf{Select}_{F_j}$ corresponds to `df["col_j"].mean()`, while $\mathsf{Mean} \circ \mathsf{Max} \circ \mathsf{GroupBy}_{F_k} \circ \mathsf{Select}_{F_j}$ correspondes to `df["col_j"].groupby(df["col_k"]).max().mean()`. Here `df` is a dataframe representing the sequence of user behaviros, e.g., $\mathcal{X}$., and operators $\mathsf{Select}, \mathsf{Mean}, \mathsf{Max}, \mathsf{GroupBy}$ are introduced as follows.

Select operators:

$$\mathbb{T}_{\text{select}} = \{\mathsf{Select}_{F_j} | j = 1, \ldots, m_1 + m_2\}.$$

The operator Select_{F_j} selects the j-th column of the data, which corresponds to `df["col_j"]` in pandas. If F_j is categorical, then values in the columns are transformed into one-hot representations, and the following operators work on each dimension of data separately. The first operator must be from this set, i.e., $\mathcal{T}_1 \in \mathbb{T}_{\text{select}}$.

Aggregation operators:

$$\mathbb{T}_{\text{agg}} = \{\mathsf{Mean}, \mathsf{Max}, \mathsf{Min}, \mathsf{Sum}, \mathsf{Mean}, \mathsf{Std}, \mathsf{Ptp}, \mathsf{Count}, \mathsf{First}\} \cup \{\mathsf{Percentile}_k | k = 5, 10, 25, 50, 75, 90, 95\}. \quad (1)$$

[1] We use the term statistic to represent a statistical function or the value of the function interchangeably.

[2] https://github.com/pandas-dev/pandas

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

As the name implies, an aggregation operator returns the corresponding statistics (Mean, Max, Min, Sum, Mean, Std, Ptp[3]) of the input column. First returns the first element, and Count returns the length of the sequence. Percentile_k computes the k-th percentile of the selected column. An aggregation operator and a select operator can compose the shortest statistics. Take $\mathcal{S}_2 = \text{Std} \circ \text{Select}_{F_k}$ in Fig. 2(a) as an example. $\mathcal{S}_2(\mathcal{X}_i)$ returns the standard deviation of the k-th column in sequence $\mathcal{X}_i$, and the pandas representation is `df["col_k"].std()`.

By operators:

$$\mathbb{T}_{\text{by}} = \{\text{GroupBy}_{F_j} \quad | j = 1, \ldots, m_1\}$$
$$\cup \{\text{FilterBy}_{F_j = f_j} \quad | j = 1, \ldots, m_1, f_j \in \mathbb{F}_j\}$$
$$\cup \{\text{RetainBy}_{F_j = f_j} \quad | j = 1, \ldots, m_1, f_j \in \mathbb{F}_j\}$$
$$\cup \{\text{SortBy}_{F_j, k} \quad | j = m_1 + 1, \ldots, m_1 + m_2, k \in \{\text{asc,desc}\}\}$$

The GroupBy_{F_j} operator partitions an input sequence into several groups according to the categorical feature F_j. A GroupBy operator must be followed by an aggregation operator that computes statistics within each group. The $\text{FilterBy}_{F_j = f_j}$ ($\text{RetainBy}_{F_j = f_j}$) operator removes (keeps) rows where the categorical feature F_j is f_j. The $\text{SortBy}_{F_j,\cdot}$ operator sorts the input sequence in ascending/descending order. This set of operators corresponds to `.groupby()`, `. filter()`, and `.sort_values()` APIs.

Transformation operators:

$$\mathbb{T}_{\text{tsfm}} = \{\text{Top}_5, \text{Abs}\}$$

The Top_5 returns the first 5 rows of the input data. The Abs returns the absolute values of the input data.

Note that our proposed method is highly extensible as users can define their own operators. A valid statistic $\mathcal{S}(\mathcal{X}_i)$ is a scalar if a numerical column is selected, otherwise a p-dimensional vector. We use $\mathbb{V}$ to denote the set of all valid statistics within the maximum depth D. For instance, $\text{Mean} \circ \text{Mean} \circ \text{Select}$ is not a valid statistic.

2) MCTS Formulation of Generating Single Statistic: Now we focus on generating one statistic alone. We formulate the statistics generation as a sequential decision process. Assume that we are going to generate a statistic $\mathcal{S}$, and we have generated a partial sequence of operators $\mathcal{T}_1, \ldots, \mathcal{T}_{i-1}$.

To determine the next optimal operator $\mathcal{T}_i$, we first initialize a search tree with a single root node v_0. Each node v (except the root) has four fields: the state $S(v)$, the action $A(v)$, the cumulative reward $Q(v)$ and the number of visits $C(v)$. The root node only has state $S(v_0) = \mathcal{T}_{i-1} \circ \cdots \circ \mathcal{T}_i$.

Then we grow the Monte Carlo search tree following the standard MCTS process. The high-level overview is that we grow the tree by repeating 1. selection: starting from the root node, the algorithm iteratively selects the most promising (partial) statistic, i.e., the sequence of operators; 2. expansion: when a non-terminal, non-fully-expanded node is reached, the algorithm expands the tree by adding a new operator to the current partial sequence of operators; 3. simulation: the algorithm performs a rollout by randomly sampling subsequent operators to form a complete statistic, and then evaluates how well this statistic explains the data; and 4. backpropagation: the algorithm updates the value of each operator in the sequence based on the evaluation, informing future choices. An example of this process is illustrated in Fig. 2(a). The steps are summarized in Algorithm 3 and Algorithm 4.

[3]Ptp: peak-to-peak

Algorithm 1: MCTS-SG Sequential Move.

Require: Dataset $\{\mathcal{X}_i\}$, teacher model's outputs $y \in \Re^N$
Initialize $\mathbb{S} = \{\}$
for $k = 1, 2, \ldots, K$ **do**
  Initialize $\mathcal{S} \leftarrow$ IdentityMapping
  for $d = 1, 2, \ldots, D$ **do**
    Initialize the root node v_0 with $S(v_0) = \mathcal{S}$
    for $it = 1, 2, \ldots, iterations$ **do**
      Get $v, \mathcal{S}' = \text{TreeGrowth}(v_0)$ ▷ Algorithm 3
      Compute the conditional association as the reward $r = \mathcal{A}(\mathcal{S}', y|\mathbb{S})$ accord. to (11) or (14)
      Backpropagation(v, r) ▷ Algorithm 4
    end for
    Select the best child v of the root accord. to (5)
    $\mathcal{T} \leftarrow A(v)$
    $\mathcal{S} \leftarrow \mathcal{T} \circ \mathcal{S}$
    if $\mathcal{S}$ is a valid statistic **then**
      break
    end if
  end for
  Mark $\mathcal{S}$ as invalid
  $\mathbb{S} \leftarrow \mathbb{S} \cup \{\mathcal{S}\}$
end for
Return $\mathbb{S}$

(Selection) From the root, we select the best child iteratively until we reach a not fully expanded node or a terminal node. A node v is not fully expanded if there exists valid operator $\mathcal{T} \in \{\mathcal{T} | \exists \mathcal{T}'_1, \ldots, \mathcal{T}'_d \text{ s.t. } \mathcal{T}'_d \circ \cdots \circ \mathcal{T}'_1 \circ \mathcal{T} \circ S(v) \in \mathbb{V}\}$ that is not the child of v. A node is terminal if its state $S(v)$ corresponds to a valid statistic. The best child is selected according to the UCT algorithm [35]:

$$\underset{v' \in Children(v)}{\arg\max} \frac{Q(v')}{C(v')} + c \cdot \sqrt{\frac{2 \ln C(v)}{C(v')}}, \tag{2}$$

where v is the current node (initialized as v_0) and $c = 1/\sqrt{2}$ is the exploration constant. The first term in (2) is the estimated state value, and the second term gives high values when a child is less frequently visited. In this case, we balance exploitation and exploration.

(Expansion) If the selected node v is not fully expanded, we pick an untried valid operator $\mathcal{T} \in \mathbb{T}$. We create a new child node v' of v with $Q(v') = C(v') = 0$, $A(v') = \mathcal{T}$, and $S(v') = \mathcal{T} \circ S(v)$. Then we return $v = v'$.

(Simulation) Now we estimate the reward of the selected node v. We denote the state of the node, i.e., statistic, as $\mathcal{S} = S(v)$. If the selected node v is not a terminal node, i.e., $S(v) = \mathcal{T}_{i+j} \circ \cdots \circ \mathcal{T}_1$ is not a valid statistic, then we randomly

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: MCTS-SG Simultaneous Move.

Require: Dataset $\{\mathcal{X}_i\}$, teacher model's outputs $y \in \Re^N$
Initialize $\mathbb{S} = \{\}$
for $k = 1, 2, \ldots, K$ **do** ▷ Initialize K trees
  Initialize $\mathcal{S}^k \leftarrow$ IdentityMapping
  Initialize the root node v_0^k with $S(v_0^k) = \mathcal{S}^k$
end for
for $d = 1, 2, \ldots, D$ **do**
  for $it = 1, 2, \ldots, iterations$ **do**
    Initialize $\mathbb{S}' \leftarrow \mathbb{S}$
    for $k = 1, 2, \ldots, K \wedge k$-th tree not finished **do**
      Get $v^k, \mathcal{S}' = \text{TreeGrowth}(v_0)$ ▷ Algorithm 3
      $\mathbb{S}' \leftarrow \mathbb{S}' \cup \{\mathcal{S}'\}$
    end for
    Compute the joint association r accord. to (7)
    Distribute the reward accord. to (8)
    for $k = 1, 2, \ldots, K \wedge k$-th tree not finished **do**
      Backpropagation(v^k, r_k) ▷ Algorithm 4
    end for
  end for
  for $k = 1, 2, \ldots, K \wedge k$-th tree not finished **do**
    Select the best child v of the root accord. to (5)
    $\mathcal{T} \leftarrow A(v)$
    $\mathcal{S}^k \leftarrow \mathcal{T} \circ \mathcal{S}^k$
    if $\mathcal{S}^k$ is a valid statistic **then**
      Mark $\mathcal{S}^k$ as invalid
      Mark the k-th tree as finished
      $\mathbb{S} \leftarrow \mathbb{S} \cup \{\mathcal{S}^k\}$
    end if
  end for
end for
Return $\mathbb{S}$

Algorithm 3. Tree Growth

Require: root node v_0
$v \leftarrow v_0$
while v is not terminal **do** ▷ Selection
  if v is not fully expanded **then** ▷ Expansion
    Choose an untried valid operator $\mathcal{T}$
    Create a child node v' of v with $Q(v') = C(v') = 0, A(v') = \mathcal{T}, S(v') = \mathcal{T} \circ S(v)$; $v \leftarrow v'$
    break
  else
    Select the child v' accord. to (2); $v \leftarrow v'$
  end if
end while
if v is terminal **then** ▷ Simulation
  $\mathcal{S} \leftarrow S(v)$
else
  Sample a valid statistic $\mathcal{S}$ accord. to (3)
end if
Return the leaf node v and the statistic $\mathcal{S}$

Algorithm 4. Backpropagation

Require: leaf node v, reward r
while v is not root **do** ▷ Backpropagation
  $C(v) \leftarrow C(v) + 1$
  $S(v) \leftarrow S(v) + r$
  $v \leftarrow parent(v)$
end while

sample operators to form a valid one:

$$\mathcal{S} \leftarrow \mathcal{T}_D \circ \cdots \circ \mathcal{T}_{i+j+1} \circ \mathcal{T}_{i+j} \circ \cdots \circ \mathcal{T}_1. \tag{3}$$

We measure the association between the statistic and the teacher model's outputs as the reward

$$r = \mathcal{A}(\mathcal{S}, y). \tag{4}$$

The measures of association will be discussed in Sections II-I-B-5 and III-B-6. Here the reward is estimated on a small batch of data to improve the efficiency of the search algorithm. During the multiple rounds of simulation on different batches, the UCT algorithm balances exploration and exploitation, which ensures that the algorithm does not overly focus on a limited part of the state space but explores sufficiently to reduce bias, and would theoretically converge to the optimal solution [11], [35].

(Backpropagation) Once we obtain the reward of the node v, we can back propagate the reward to its ancestors. Specifically, do the following updates until we reach the root node:

$$C(v) \leftarrow C(v) + 1,$$
$$Q(v) \leftarrow Q(v) + r,$$
$$v \leftarrow parent(v).$$

After a certain number of iterations, we can terminate the tree growing algorithm and return the best child of the root:

$$v = \underset{v' \in Children(v_0)}{\arg\max} \frac{Q(v')}{C(v')}, \tag{5}$$

We update the partial solution with $\mathcal{T}_i = A(v)$. Then we create a new search tree with the root $\mathcal{T}_i \circ \cdots \circ \mathcal{T}_1$ and repeat the algorithm until we construct a valid statistic.

So far we have described the algorithm for generating one statistic. Can we invoke the algorithm K times to generate K statistics? Apparently no. The algorithm's objective is to maximize the association between the statistic and the teacher model's outputs. If we naively run the algorithm multiple times, we are likely to obtain multiple highly relevant and redundant statistics, for example, the mean, the sum, and the 50th percentile of "amount" in Fig. 1.

So in the following subsections, we introduce the sequential move game and the simultaneous move coalitional game to improve the generation of multiple statistics.

3) Sequential Move Game: In the sequential move game, we generate statistics one after another.

Assume that we have generated k statistics $\mathbb{S}_k = \{\mathcal{S}_1, \ldots, \mathcal{S}_k\}$. To generate the $(k+1)$-th statistic, we consider

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

the following conditional association function as the reward in (4):

$$r = \mathcal{A}(\mathcal{S}_{k+1}, y|\mathbb{S}_k). \tag{6}$$

Moreover, once we obtain the $\mathcal{S}_{k+1}$, we mark it invalid by updating $\mathbb{V} \leftarrow \mathbb{V}\backslash\{\mathcal{S}_k\}$. In this case, it would not be generated again even in simulation processes (3) and further reduce the redundancy of statistics.

4) Simultaneous Move Coalitional Game: In the simultaneous move coalitional game, we generate K statistics simultaneously.

Specifically, we adopt the simple but effective decoupled UCT [36] algorithm. We initialize K search trees and grow the trees synchronously. Each tree selects its own move independently, and at the simulation step, all trees form a coalition of statistics $\mathbb{S} = \{\mathcal{S}_1, \ldots, \mathcal{S}_K\}$. However, different from two-player zero-sum games widely studied in the literature on simultaneous move games [12], [37], when generating statistics, all trees can only obtain a joint reward

$$r = \mathcal{A}(\mathbb{S}, y). \tag{7}$$

In order to distribute the joint reward to each tree, we consider the Shapley values [13] to measure the marginal contribution of each statistic. Given a subset of statistics $\mathbb{C} \subseteq \mathbb{S}$ not including $\mathcal{S}_i$, we can easily compute the marginal importance of statistic $\mathcal{S}_i$ by comparing the degrees of association

$$\Delta(\mathbb{C}, \mathcal{S}_i) = \mathcal{A}(\mathbb{C} \cup \{\mathcal{S}_i\}, y) - \mathcal{A}(\mathbb{C}, y).$$

Then the Shapley value-based reward can be computed by averaging over all possible subsets:

$$r_i = \sum_{\mathbb{C}\subseteq\mathbb{S}\backslash\{\mathcal{S}_i\}} \frac{|\mathbb{C}|!(K-1-|\mathbb{C}|)!}{K!} \Delta(\mathbb{C}, \mathcal{S}_i). \tag{8}$$

In this way, each tree searches a statistic that can maximize its own marginal contribution under all possible coalitions, i.e., combinations of statistics. Due to the exponential computational complexity, we follow the common practice [13] to estimate (8) by sampling.

5) Correlation-Based Measure of Association: In this paper, we consider a correlation-based measure of association $\mathcal{A}_{\text{corr}}$, and a causality-based measure $\mathcal{A}_{\text{causal}}$.

First, if we assume all relationships are linear, we could design the association measure as the (absolute) correlation between the statistic values and the teacher model's outputs

$$\mathcal{A}_{\text{corr}}(\mathcal{S}, y) = |\text{corr}(x, y)|, \tag{9}$$

where $x = (\mathcal{S}(\mathcal{X}_1), \ldots, \mathcal{S}(\mathcal{X}_N))^\top \in \Re^N$ is the value of the statistic $\mathcal{S}$. And $\text{corr}(x, y) = \frac{\bar{x}^\top \bar{y}}{\sqrt{\bar{x}^\top \bar{x} \cdot \bar{y}^\top \bar{y}}}$ is the Pearson correlation coefficient, where $\bar{x}, \bar{y}$ are centralized x, y.

If $\mathcal{S}$ acts on a categorical feature, i.e., its Select operator selects a categorical feature, the output is denoted as

$$X = (\mathcal{S}(\mathcal{X}_1), \ldots, \mathcal{S}(\mathcal{X}_N))^\top \in \Re^{N\times p}.$$

Then we define the reward as the absolute value of the coefficient of multiple correlation [38]:

$$\mathcal{A}_{\text{corr}}(\mathcal{S}, y) = |\text{corr}(y', y)|,$$

$$\text{where} \quad \beta, \beta_0 = \arg\min_{\beta\in\Re^p, \beta_0\in\Re} \|y - X \cdot \beta + \beta_0\|_2,$$
$$y' = X \cdot \beta + \beta_0. \tag{10}$$

In this case we compute the Pearson correlation coefficient between the actual values and the fitted values of a linear regression model. The above (10) is also used when compute the joint reward (7), in which case the X represents the concatenation of multiple statistics values.

To compute the association conditioning on $\mathbb{S}_k$, we consider the partial correlation by computing the correlation between the statistics and the residuals of the linear regression.

$$\mathcal{A}_{\text{corr}}(\mathcal{S}, y|\mathbb{S}_k) = \mathcal{A}_{\text{corr}}(X, y - (Z \cdot \beta' + \beta'_0))$$
$$\text{where} \quad \beta'_0, \beta' = \arg\min \|y - (Z \cdot \beta' + \beta'_0)\|_2. \tag{11}$$

Here, Z is the concatenation of values of previously generated statistics:

$$Z = (X_1, \ldots, X_k) \in \Re^{N\times P}, \tag{12}$$

where P is the corresponding dimension after concatenation, and X_i, the value of the i-th statistic, is with dimension $\Re^N$ or $\Re^{N\times p}$ depending on whether it acts on a categorical column.

Now the objective of the algorithm is to maximize the (absolute) partial correlation between the statistic and the outputs of the teacher, given existing generated statistics. In this case, we reduce the collinearity of statistics.

6) Causality-Based Measure of Association: In general, the correlation-based measures only consider linear cases and does not capture the causal relationships. Recent studies [3] have shown that causal features (statistics) can improve the interpretability and the robustness of downstream models. Thus we also propose a causality-based measure of association $\mathcal{A}_{\text{causal}}$. Following the definition in previous literature [3], we say a feature (statistic) is with casual relationship with the target variable, i.e., the teacher model's outputs, if they are dependent conditioning on previously generated statistics. Thus the key is to filter out (conditional) independent statistics with some tests [22], [24], [25].

For unconditional cases in (4) and 7, we adopt the Hilbert-Schmidt independence criterion (HSIC) [25] to measure the dependency between two variables:

$$HSIC(X, Y) = \text{Tr}(\mathbf{K}_X \cdot \mathbf{K}_y),$$

where Tr denotes the matrix trace, X and Y are two matrices (vectors) with N rows, and $\mathbf{K}_X, \mathbf{K}_Y \in \Re^{N\times N}$ are the centralized kernel matrices of X and Y respectively. $HSIC(X, Y) = 0$ implies that X and Y are independent. Here we use the standard Gaussian kernel.

We use the normalized HSIC as the reward function:

$$\mathcal{A}_{\text{causal}}(\mathcal{S}, y) = \frac{HSIC(X, y)}{\sqrt{HSIC(X, X) \cdot HSIC(y, y)}}. \tag{13}$$

The above (13) is also used when compute the joint reward (7), in which case the X represents the concatenation of multiple statistics values.

For conditional cases, we adopt the kernel-based conditional independence criterion (KCI) [24] as it is a generalization of

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

HSIC:

$$KCI(X, y|Z) = \text{Tr}(\mathbf{K}_{\ddot{X}|Z} \cdot \mathbf{K}_{y|Z}),$$

$$\text{where} \quad \ddot{X} = (X, Z), \quad \text{(concatenation)}$$

$$\mathbf{K}_{\ddot{X}|Z} = R_Z \cdot \mathbf{K}_{\ddot{X}} \cdot R_Z,$$

$$\mathbf{K}_{y|Z} = R_Z \cdot \mathbf{K}_Z \cdot R_Z,$$

$$R_Z = \epsilon(\mathbf{K}_z + \epsilon I)^{-1}.$$

Here, we use $\epsilon = 0.001$ as a small regularization parameter. $\mathbf{K}_{\ddot{X}}, \mathbf{K}_y, \mathbf{K}_Z \in \Re^{N\times N}$ are the corresponding centralized kernel matrices. $KCI(X, y|Z) = 0$ indicates that X and y are conditional independent, given Z.

We use the normalized version as the measure of association between X and y, conditioning on Z:

$$\mathcal{A}_{\text{causal}}(\mathcal{S}, y|\mathbb{S}_k) = \frac{\text{Tr}(\mathbf{K}_{\ddot{X}|Z} \cdot \mathbf{K}_{y|Z})}{\sqrt{\text{Tr}(\mathbf{K}_{\ddot{X}|Z} \cdot \mathbf{K}_{\ddot{X}|Z}) \cdot y(\mathbf{K}_{y|Z} \cdot \mathbf{K}_{y|Z})}}, \tag{14}$$

where Z is the concatenation of values of previously generated statistics as (12). By using the KCI-based reward function in the sequential move game, we filter out statistics that are conditionally independent of target y, given the previously generated statistics.

C. Stage 2: Rule Distillation With Neural Logical Network

In this section, we describe how to distill rules from the teacher model with the generated statistics. As demonstrated in Fig. 2(b), our proposed GS-NLN contains an input layer, several logical layers, and a prediction layer.

1) Input Layer: Assume that we have obtained the values of top-K statistics:

$$X = (X_1, \ldots, X_K) \in \Re^{N\times P},$$

where P is the corresponding dimension after concatenation. Some columns of X have already been binary, e.g., categorical features with Max operator, while the others have not. So we iterate over the columns of X. If a column corresponds to a continuous statistic, e.g., the "average amount", we binarize it according to its percentiles:

$$x'_j = \begin{cases} x_j & \text{if } x_j \in \{0,1\}^N, \\ (\mathbb{I}_{x_j > x_j^0}, \ldots, \mathbb{I}_{x_j > x_j^{90}}, \mathbb{I}_{x_j > p_j^{100}}) \in \{0,1\}^{N\times 11} & \text{otherwise.} \end{cases} \tag{15}$$

Here, $\{x_j^k | k = 0, \ldots, 90, 100\}$ are percentiles of x_j, $\mathbb{I}_{\text{cond}}$ is the element-wise indicator function that returns 1 if cond is true, otherwise 0. We denote the binarized features as $X' = (x'_1, \ldots, x'_P) \in \{0,1\}^{N\times P'}$, and each element in X' corresponds to a boolean literal, for example, "average amount $<$ \$1.00".

Now let us focus on a certain row of X', i.e., the boolean representations of the i-th user, denoted as $z_i = \text{row}_i(X')^\top \in \{0,1\}^{P'}$. We enhance the input by

$$z_i^0 = (z_i^\top, (1 - z_i)^\top, 1, 0)^\top \in \{0,1\}^{h_0},$$

where $h_0 = 2 \cdot P' + 2$. Since the following logical layers are only cable of doing AND/OR operations, we enhance the input by appending its negations. Moreover, extra 1 and 0 are added to enable the following layers to learn shortcuts.

2) Logical Layers: Our proposed GS-NLN has L logical layers, and each is composed of a conjunction part and a disjunction part. Let $z_i^{l-l} \in \{0,1\}^{h_{l-1}}$ denote the binary hidden representation after the $(l-1)$-th layer. The j-th neuron in the conjunction part would select two elements in z_i^{l-l} by sampling from the parameterized categorical distributions:

$$q_j^{l,k} \sim \text{onehot}\left(\text{softmax}(w_j^{l,k})\right), \; k = 1, 2 \tag{16}$$

where $w_j^{l,k} \in \Re^{h_{l-1}}$ are parameters. Then the value of the neuron is the boolean AND between the input values:

$$u_j^l = (q_j^{l,1})^\top z_i^{l-1} \cdot (q_j^{l,2})^\top z_i^{l-1} \in \{0,1\},$$

Similarly, neurons in the disjunction part conduct the boolean OR operations:

$$v_j^l = 1 - (1 - (q_j^{l,3})^\top z_i^{l-1}) \cdot (1 - (q_j^{l,4})^\top z_i^{l-1}) \in \{0,1\},$$

$$q_j^{l,k} \sim \text{onehot}\left(\text{softmax}(w_j^{l,k})\right), \; k = 3, 4. \tag{17}$$

The sampling operations in (16) and (17) cause the network no longer differentiable. So we resort to the Gumbel-softmax estimator [14], and re-write them into

$$q_j^{l,k} = \text{softmax}((w_j^{l,k} + g_k) \cdot \tau^{-1}), \; k = 1, 2, 3, 4, \tag{18}$$

where $g_k \in \Re^{h_{l-1}}$ are h_{l-1} i.i.d. samples from the Gumbel distribution $\text{Gumbel}(0, 1)$, and τ is the temperature. As τ approaches 0, (18) becomes identical to (16) and (17). During training, we start at a high temperature $\tau = 1$ and anneal to a low temperature, e.g., $\tau = 0.0001$. Different from previous logical activation functions that are prone to vanishing gradients and thus require more training tricks [5], [15], our formulation ensures that each neuron in the network connects to only two neurons from the previous layer with atomic AND/OR operations, making the neural network easy to train.

The output of the layer is composed of

$$z^l = (u_1, \ldots, u_H, v_1, \ldots, v_H, (z^{l-1})^\top)^\top \in \{0,1\}^{h_l},$$

where $h_l = 2 \cdot H + h_{l-1}$, and H is the hidden size. We also use skip connections so that the model can adjust the complexity of rules adaptively. But at the last layer, we do not use the skip connection and use $\frac{R}{2}$ neurons in conjunction and disjunction parts respectively, where R is the number of desired rules.

3) Prediction Layer: After L logical layers, we obtain $z_i^L \in \{0,1\}^R$ where each element represents a logical rule. Now we get the score of the instance by a linear prediction layer $\tilde{y}_i = w^\top z_i^L$, where $w \in \Re^R$ is the model parameter representing the weights of rules. It is possible to pass w or $\tilde{y}_i$ through non-linear functions. For example, we could use the softplus function if we want the weights of rules nonnegative, or use sigmoid function

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

to constrain scores to be with range (0, 1). We leave this for future work.

4) Model Training: The training objective is to mimic the outputs of the teacher model. We use the following ranking-based loss function

$$\mathcal{L} = \frac{1}{N \times (N-1)} \sum_{i \neq j} \left(\mathbb{I}_{y_i > y_j} \ln \sigma(\tilde{y}_i - \tilde{y}_j) + \mathbb{I}_{y_i \leq y_j} \ln \sigma(\tilde{y}_j - \tilde{y}_i) \right),$$

where $\sigma(\cdot)$ is the sigmoid function. In practice, we use the stochastic gradient descent to optimize the loss function, and an instance i is only compared to the samples in the same mini-batch.

5) Differences Between GS-NLN and RRL: RRL [5] is the state-of-the-art algorithm that learns logical rules. Though our proposed GS-NLN and RRL have similar outputs, they are highly different. RRL aims to learn rule-based representations for interpretable classification, while GS-NLN is to distill the knowledge of a teacher into a rule system. So in GS-NLN, a neuron can only connect to two input units to maintain the low complexity of the rules. This design ensures that the learnt rules are simple and easy to use. For example, rules produced by an one-layer GS-NLN could only be in the form of "A AND/OR B", and rules by a two-layer model follow the form of "(A AND/OR B) AND/OR (C AND/OR D)". On the contrary, in RRL a neuron can connect to any number of units for better performance.

Another difference lies in the way discrete activations are handled in networks. In RRL, an extra continuous version of the network is maintained, where boolean operations are implemented with logical activation functions [15]. The authors proposed the gradient grafting technique, and they use the grafted gradients of the continuous network to guide the updates of the discrete one. In our work, since each neuron only connects to two units, we would hardly face notorious vanishing gradient problems of logical activation functions, and straightforward Gumbel-softmax estimators can learn parameters very well.

D. Complexity

To efficiently measure the relevancy between the statistics and the teacher model's outputs during the tree search, we sample mini-batches of data with size B. In this case, in (6), (10), and (11), the values of statistics and the target y are only with B rows. And kernel matrices in (13) and (14) are with shape $\Re^{B \times B}$. During the tree search, there are $K \times D \times B \times$ (number_of_simulations) data points being evaluated, which is independent of the dataset size n or the space of statistics. Since the MCTS has the *anytime* property [11], the number of simulations could be arbitrarily large as long as it's within the time constraint, and thus the overall time complexity is linear w.r.t the number of simulations. We use MCTS to explore the space of statistics systematically, and Algorithm 1 and Algorithm 2 are highly scalable. One benefit of the simultaneous game formulation (Algorithm 2) is that K trees can be parallelized straightforwardly, which may further reduce the running time.

For the GS-NLN, the first logical layer transforms $O(K)$ statistics into $O(H)$ hidden neurons, while the subsequent layers generate new $O(H)$ hidden neurons with previous neurons. The final output layer generate R rules from $O(H)$ hidden neurons. Since all these operations are achieved by vector-matrix multiplication, the overall complexity of the GS-NLN is $O(K \times H + H^2 \times (L-1) + H \times R)$.

TABLE I
STATISTICS OF DATASETS

	VEWS	Elo	RedHat	Industrial
# Users	33,576	201,917	144,639	222,306
# Records	772,530	19,249,694	1,889,213	34,722,334
# Cat. Features	7	12	11	6
# Num. Features	0	1	0	1

IV. EXPERIMENTS

A. Datasets and Teacher Models

We use 3 public datasets: (1) VEWS [39] contains 33,576 users and their edit behaviors on Wikipedia, among which Wikipedia administrators blocked 17,027 users for vandalism. (2) Elo[4] contains transactions of 201,917 users, and 97,609 users are marked positive due to their customer loyalty. (3) RedHat[5] contains activities of 144,639 users, and 62,115 users are positive considering their potential business values. An extra industrial dataset is analyzed in Section IV-E. The statistics of the datasets are summarized in Table I.

Two models are used as teachers: (1) GRU: We use a bi-directional 2-layer GRU with 128 hidden units and dropout ratio 0.2. (2) LightGBM: As one of the most popular methods in data mining competitions, we use LightGBM [10] as the second teacher model. We use all aggregation operators introduced in (1) to transform raw sequences into tabular forms.

We use 80% users' sequences as the training data, among which 1,000 sequences are used for validation. The remaining are used for testing. We train GRU and LightGBM with early stopping.

Unless otherwise stated, we use the default hyper-parameters of CauseRuDi summarized in Table II. Experiments on public datasets were conducted on a single server with 72 cores, 128 GB memory, and four Nvidia Tesla V100 GPUs.

B. Experiment 1: Statistics Generation

In the first group of experiments, we test if our proposed MCTS-SG can generate informative statistics. We use *seq* and *sim* to denote the sequential game formulation Algorithm 1 and the simultaneous game formulation Algorithm 2 respectively. We use *corr* and *causal* to denote the measures of association. Note that the RuDi [40] corresponds to the special form of MCTS-SG with *seq* and *corr*.

As we reviewed in Section II, most automatic feature engineering methods cannot be directly applied to behavioral sequences. We consider the following alternative approaches. (1)

[4]https://www.kaggle.com/c/elo-merchant-category-recommendation/
[5]https://www.kaggle.com/c/predicting-red-hat-business-value/

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DEFAULT VALUES OF HYPER-PARAMETERS IN CAUSERUDI

Hyper-parameter	Value
(MCTS-SG)	
Statistics Depth D	4
Number of statistics K	20
Batch size B	128
Number of simulations	500
(GS-NLN)	
Number of rules R	20
Hidden size H	20
Number of logical layers L	2
Optimizer	Adam
Batch size	128
Epochs	500
Learning rate	$0.1 \rightarrow 0.001$ (linear decay)
Temperature τ	$1.0 \rightarrow 0.0001$ (linear decay)

Random: We generate K statistics by randomly composing operators introduced in Section III-B-1. (2) Lasso [41]: We apply all aggregation operators (1) on the datasets to transform sequences into fixed-length feature vectors, then train Lasso models to perform feature selection. (3) LightGBM [10]: Similarly, we train LightGBM on the flattened feature vectors and then select features with the highest importance scores. (4) OneBM [21]: This method can generate features on relational databases and use Chi-squared testing to select features. We transform user sequences into two relational tables to meet its input requirements. Other feature generation methods on relational databases like DSM [20] have been shown inferior to OneBM and thus are omitted here.

Since we use LightGBM as a baseline here, we choose GRU as the teacher model. To measure how informative the generated statistics are, we measure the linear relevance as well as the non-linear dependence between statistics/features and the teacher model's outputs on the training splits. In linear cases, we measure the coefficients of multiple correlation as (10), denoted as *corr*. In non-linear cases, we consider the distance correlation [42], [43] as the metric function *dcorr*. The distance correlation is a statistical measure that captures both linear and nonlinear associations between two random variables. It is computed by first determining the distance covariance between the variables, and then normalizing this value by the product of their distance standard deviations. Due to the computational complexity of the distance correlation, we compute *dcorr* scores on 10,000 random samples using the package `dcor`.[6] Both *corr* and *dcorr* range from 0 to 1. The higher the values are, the more informative the generated statistics are.

The results are shown in Table III. As we can see, all variants of MCTS-SG outperform baselines significantly. Lasso, LightGBM and OneBM cannot generate deep statistics as MCTS-SG and thus only achieve suboptimal performance. Though Random can generate deep statistics, the exponentially large search space makes it impossible to generate useful statistics without systematic search strategies. These results prove the effectiveness of our MCTS-based statistics generation method.

[6]https://github.com/vnmabus/dcor

By comparing the results of linear and non-linear cases, we can observe that the causality-based reward functions indeed guide the tree to search causal statistics. With the correlation-based reward, the sequential move version is slightly better than the simultaneous one. This may be due to the variance in estimating Shapley values [13]. On the contrary, with the causality-based reward, the sequential move game formulation shows inferior performance. We ascribe this phenomenon to the unreliable results of conditional independence tests when conditioning sets are large [3], [24].

C. Experiment 2: Rule Distillation

Now we show the results of rule distillation tasks where the goal is to distill the knowledge of teacher models into rule-based systems.

To the best of our knowledge, this paper is the first study on sequence model explanation. There are no out-of-the-box baseline models, and most of them require fixed-length feature vectors as inputs. In the first group of experiments, we have shown that MCTS-SG can generate informative statistics. So we include the following alternatives with inputs being statistics generated by MCTS-SG: (1) Lasso: We train Lasso models to mimic the outputs of teacher models and tune penalty coefficients so that the number of nonzero weights is approximately R. (2) Lasso (Binary): Considering Lasso can only assign weights to the features and is not a real rule model, we use the binarized statistics (15) as input features. (3) CART [44]: This is a classic decision tree model that can learn hierarchical rules. (4) RRL [5]: This is the state-of-the-art neural logical model we have introduced before (Section III-C-5). Other logical models like SBRL [7], CORELS [6], CRS [4] have been shown inferior to RRL and thus are not included. For linear models Lasso, Lasso (Binary), and our model CauseRuDi (corr), we use statistics generated by MCTS-SG (seq, corr); for non-linear models CART, RRL, and our model CauseRuDi (causal), we use statistics generated by MCTS-SG (sim, causal).

We show AUC scores to test the performance of distilled rule models. Also, we use the following score to test if the distilled models are with fidelity to their teachers.

$$\text{Fidelity} = \frac{1}{N \times (N-1)} \sum_{i \neq j} \mathbb{I}_{y_i > y_j} \odot \mathbb{I}_{\tilde{y}_i > \tilde{y}_j},$$

where $\odot$ is the logical XNOR operation. The higher the value, the more faithful the student.

We train comparing models on the training splits, and results on testing splits are summarized in Table IV. We can observe that our proposed two-stage methods CauseRuDi achieve the best performance constantly, and in most cases CauseRuDi (causal) shows better results. As our proposed neural logical network is a non-linear model, the statistics extracted by the MCTS-SG with causality-based reward cope better with the GS-NLN. RRL achieves comparable results, but the most complicated rule has 40 literals, which reduces its interpretability. On the contrary, our proposed CauseRuDi with two logical layers produces neat rules with at most 4 literals.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EXPERIMENTAL RESULTS OF STATISTICS GENERATION

	corr			dcorr		
	VEWS	Elo	RedHat	VEWS	Elo	RedHat
Random	0.7048	0.4198	0.5906	0.1211	0.0992	0.1905
Lasso	0.9302	0.6363	0.6229	0.2949	0.2949	0.3122
LightGBM	0.9240	0.6769	0.6292	0.4646	0.3497	0.2970
OneBM	0.9120	0.6868	0.6555	0.4642	0.3583	0.3316
MCTS-SG (seq, corr)	**0.9429**	**0.7262**	**0.7872**	0.6895	0.4005	0.2140
MCTS-SG (sim, corr)	0.9404	0.7139	0.7734	0.6738	0.4123	0.2221
MCTS-SG (seq, causal)	0.9211	0.6716	0.7035	**0.7239**	0.4428	0.3858
MCTS-SG (sim, causal)	0.9242	0.6660	0.7096	0.7130	**0.4439**	**0.3945**

TABLE IV
EXPERIMENTAL RESULTS OF RULE DISTILLATION

	VEWS		Elo		RedHat	
	Fidelity	AUC	Fidelity	AUC	Fidelity	AUC
(Teacher: GRU)		(0.9764)		(0.6345)		(0.6381)
Lasso	0.8863±0.0001	0.9637±0.0001	0.6828±0.0003	0.5491±0.0001	0.7347±0.0011	0.5828±0.0003
Lasso (Binary)	0.8874±0.0001	0.9615±0.0001	0.7166±0.0001	0.5763±0.0001	0.7249±0.0023	0.5769±0.0008
CART	0.8395±0.0001	0.9496±0.0001	0.7209±0.0001	0.5775±0.0001	0.7719±0.0001	0.5959±0.0001
RRL	0.8058±0.0023	0.9304±0.0049	0.7100±0.0022	0.5665±0.0007	0.7728±0.0025	0.5930±0.0053
CauseRuDi (corr)	0.9206±0.0020	**0.9666**±0.0006	0.7625±0.0004	0.5807±0.0007	0.8007±0.0020	0.6095±0.0013
CauseRuDi (casual)	**0.9299**±0.0013	0.9657±0.0042	**0.7710**±0.0002	**0.5832**±0.0004	**0.8152**±0.0065	**0.6187**±0.0041
(Teacher: LightGBM)		(0.9726)		(0.6211)		(0.6895)
Lasso	0.8865±0.0007	0.9673±0.0001	0.8619±0.0001	0.6114±0.0001	0.6881±0.0008	0.5883±0.0001
Lasso (Binary)	0.8721±0.0005	0.9647±0.0001	0.8789±0.0004	0.6135±0.0001	0.6870±0.0022	0.5868±0.0011
CART	0.8409±0.0001	0.9477±0.0001	0.8954±0.0001	0.6155±0.0001	0.7002±0.0001	0.5923±0.0001
RRL	0.8215±0.0066	0.9234±0.0091	0.8680±0.0039	0.6096±0.0017	0.7131±0.0035	0.6097±0.0067
CauseRuDi (corr)	0.9161±0.0004	0.9680±0.0003	0.9048±0.0004	0.6181±0.0006	**0.7464**±0.0032	**0.6198**±0.0017
CauseRuDi (causal)	**0.9235**±0.0001	**0.9682**±0.0008	**0.9155**±0.0001	**0.6210**±0.0011	0.7375±0.0012	0.6156±0.0024

D. Parameter Sensitivity

In this section, we study the importance of several model parameters with GRU being the teacher, and show results in Fig. 3.

First we study how the max depth of statistics D affects the quality of generated statistics. We use MCTS-SG (seq, corr) and present the coefficients of multiple correlation in Fig. 3(a), and the average depth of generated statistics in Fig. 3(b). We can observe that on Elo dataset, as we increase the max depth, the generated statistics become deeper and result in better performance. On the contrary, on VEWS and RedHat, the generated statistics are relatively shallow even the max depth D is large. This means that our proposed statistics generation method can adaptively adjust the complexity of statistics according to datasets thanks to MCTS-based search strategy.

We also study the importance of model parameters by varying the number of statistics K, the number of rules R, the number of layers L, and the hidden size H. From Fig. 3(c) we can observe that more statistics would lead to better performance. On VEWS and Elo datasets, it seems 20 statistics are enough, while on RedHat a better result may be achieved by generating more statistics. The number of rules (Fig. 3(d)) also positively impacts the performance. CauseRuDi is insensitive to the number of layers and hidden size as shown in Fig. 3(e) and (f). These results inspire us to achieve better results with more statistics and rules (larger K and R) and maintain the low complexity of each rule (small L and H).

E. Case Study

We do a case study on a sampled industrial dataset in this subsection. Tencent Mobile Payment Dataset is a private real-world financial fraudulent detection dataset from the Tencent Mobile Payment.[7] The dataset includes 34.7 million transactions from 222,306 users. Less than 4.56% of users are labeled as fraudsters. The teacher model is an ensemble model with manually-designed features by senior business analysts. We use the sequential move game formulation with the correlation-based reward to generate statistics. A part of distilled rules is summarized in Table V. Clearly, CauseRuDi successfully explains the black-box scoring model with a rule-based system, and rules are highly aligned with the industrial scenarios.

V. APPLICATIONS AND DISCUSSION

In this paper, we propose CauseRuDi, a two-stage framework that distills black-box behavior sequence models into rule models. The distilled rules are interpretable and transparent, and reveal the underlying mechanisms of teacher models. There are two potential use cases of CauseRuDi.

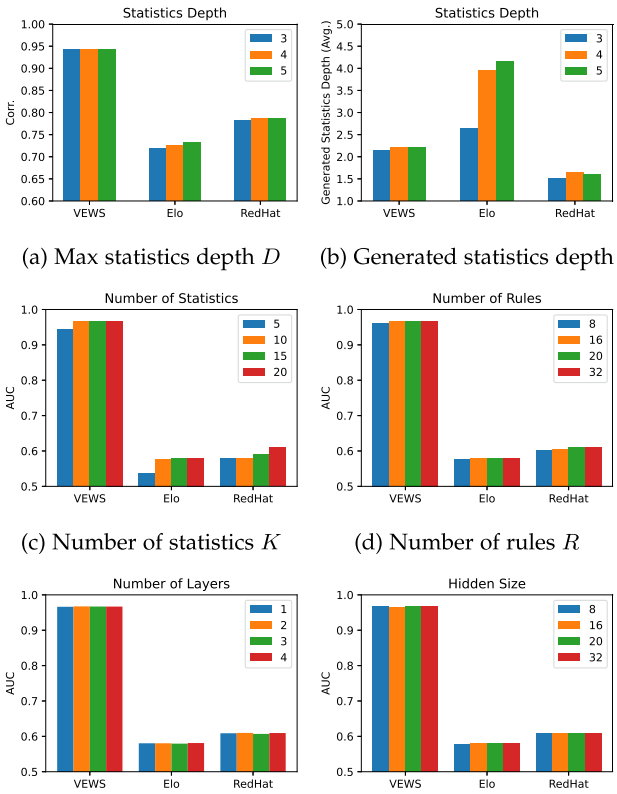
Integration with existing systems: The distilled rules can be integrated into existing rule-based systems, supplementing them

[7]The dataset is desensitized and sampled properly only for experiment purpose, and does not imply any commercial information. All personal identity information (PII) has been removed. Besides, the experiment was conducted locally on Tencent's server by formal employees.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

TABLE V
THE LEARNED RULES AND THEIR MEANINGS ON INDUSTRIAL DATASET

Weight	Rule	Meaning
-0.3722	$(\text{Sum} \circ \text{Select}_{\text{cardtype=debit}} > 14)$ OR $(\text{Sum} \circ \text{Select}_{\text{money}} > 44916)$	Users with over 14 debit card transactions or 44916 total payment amount are less likely to be fraudulent.
-0.3042	$\text{Sum} \circ \text{Select}_{\text{cardtype=credit}} > 9$	Users with over 9 credit card transactions are less likely to be fraudulent.
+0.5174	$(\text{Sum} \circ \text{Select}_{\text{industry=virtual}} > 9)$ AND $(\text{NOT Sum} \circ \text{Select}_{\text{industry=household}} > 17)$	Users with over 9 transactions in virtual products and no more than 17 transactions in household products are more likely to be fraudulent.
+0.4564	$(\text{NOT Sum} \circ \text{Select}_{\text{money}} > 4457.3)$ AND $(\text{NOT Mean} \circ \text{Select}_{\text{scene=travel}} > 0)$	Users with no more than 4457.3 total payment amount and no travel-related transactions are more likely to be fraudulent.



(a) Max statistics depth D (b) Generated statistics depth

(c) Number of statistics K (d) Number of rules R

(e) Number of layers L (f) Hidden size H

Fig. 3. Parameter study.

with new insights and trends captured by the more complex black-box models. There might be some of the extracted rules already known to the human expert, which could be used to verify if the rule system is consistent with prior beliefs [45].

Model auditing [29]: In conservative businesses like banking, where regulatory compliance is mandatory, CauseRuDi can provide the necessary explanations for the decisions made by machine learning models. It allows auditors and compliance officers to verify that the models are making decisions based on legitimate and non-discriminatory factors.

VI. CONCLUSION

In this paper, we propose CauseRuDi, which can distill the knowledge of a user behavior sequence model into a set of rules. An MCTS-based statistic generation method, MCTS-SG, is proposed to extract informative statistics from behavior sequences. To generate multiple statistics, we formulate the sequential move game and the simultaneous move game. A correlation-based reward function and a causality-based reward function are also proposed. Then a novel neural logical model, GS-NLN, is utilized to compose rules from the statistics. Experiments on three public datasets and one industrial dataset prove the effectiveness of our method.

As for future work, we will explore if the proposed MCTS-based search method can support extracting features/statistics from relational databases. Moreover, the general idea of the sequential move game is similar to the forward selection phase of forward-backward causal feature selection methods [3]. It is also possible to incorporate the backward elimination phase to remove unsatisfying statistics generated in the early steps.

REFERENCES

[1] S. Yuan, P. Zheng, X. Wu, and Q. Li, "Insider threat detection via hierarchical neural temporal point processes," in *Proc. IEEE Int. Conf. Big Data*, 2019, pp. 1343–1350.
[2] S. Yuan, P. Zheng, X. Wu, and H. Tong, "Few-shot insider threat detection," in *Proc. Conf. Inf. Knowl. Manage.*, 2020, pp. 2289–2292.
[3] K. Yu et al., "Causality-based feature selection: Methods and evaluations," *ACM Comput. Surv.*, vol. 53, no. 5, pp. 1–36, 2020.
[4] Z. Wang, W. Zhang, L. Ning, and J. Wang, "Transparent classification with multilayer logical perceptrons and random binarization," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 6331–6339.
[5] Z. Wang, W. Zhang, N. Liu, and J. Wang, "Scalable rule-based representation learning for interpretable classification," in *Proc. Conf. Neural Inf. Process. Syst.*, 2021, pp. 30479–30491.
[6] E. Angelino, N. Larus-Stone, D. Alabi, M. Seltzer, and C. Rudin, "Learning certifiably optimal rule lists for categorical data," *J. Mach. Learn. Res.*, vol. 18, pp. 234:1–234:78, 2017.
[7] H. Yang, C. Rudin, and M. Seltzer, "Scalable Bayesian rule lists," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3921–3930.
[8] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, pp. 1157–1182, 2003.
[9] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int. J. Comput. Vis.*, vol. 129, no. 6, pp. 1789–1819, 2021.
[10] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Conf. Neural Inf. Process. Syst.*, 2017, pp. 3146–3154.
[11] C. B. Browne et al., "A survey of monte carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, Mar. 2012.
[12] M. J. Tak, M. Lanctot, and M. H. Winands, "Monte Carlo tree search variants for simultaneous move games," in *Proc. IEEE Conf. Comput. Intell. Games*, 2014, pp. 1–8.
[13] E. Štrumbelj and I. Kononenko, "Explaining prediction models and individual predictions with feature contributions," *Knowl. Inf. Syst.*, vol. 41, no. 3, pp. 647–665, 2014.
[14] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–12.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

[15] A. Payani and F. Fekri, "Learning algorithms via neural logic networks," 2019, arXiv: *1904.01554*.

[16] U. Khurana, D. Turaga, H. Samulowitz, and S. Parthasrathy, "Cognito: Automated feature engineering for supervised learning," in *Proc. IEEE Int. Conf. Data Mining*, 2016, pp. 1304–1307.

[17] G. Katz, E. C. R. Shin, and D. Song, "ExploreKit: Automatic feature generation and selection," in *Proc. IEEE Int. Conf. Data Mining*, 2016, pp. 979–984.

[18] M. Song, J. Wang, T. Zhang, G. Zhang, R. Zhang, and S. Su, "Effective automated feature derivation via reinforcement learning for microcredit default prediction," in *Proc. Int. Joint Conf. Neural Netw.*, 2020, pp. 1–8.

[19] G. Zhu, Z. Xu, X. Guo, C. Yuan, and Y. Huang, "DIFER: Differentiable automated feature engineering," 2020, arXiv: *2010.08784*.

[20] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. IEEE Int. Conf. Data Sci. Adv. Analytics*, 2015, pp. 1–10.

[21] H. T. Lam, J.-M. Thiebaut, M. Sinn, B. Chen, T. Mai, and O. Alkan, "One button machine for automating feature engineering in relational databases," 2017, arXiv: *1706.00327*.

[22] C. Heinze-Deml, J. Peters, and N. Meinshausen, "Invariant causal prediction for nonlinear models," *J. Causal Inference*, vol. 6, no. 2, 2018, Art. no. 20170016.

[23] J. M. Hofman, A. Sharma, and D. J. Watts, "Prediction and explanation in social systems," *Science*, vol. 355, no. 6324, pp. 486–488, 2017.

[24] K. Zhang, J. Peters, D. Janzing, and B. Schölkopf, "Kernel-based conditional independence test and application in causal discovery," in *Proc. 27th Conf. Uncertainty Artif. Intell.*, 2011, pp. 804–813.

[25] A. Gretton, O. Bousquet, A. Smola, and B. Schölkopf, "Measuring statistical dependence with Hilbert-Schmidt norms," in *Proc. Int. Conf. Algorithmic Learn. Theory*, Springer, 2005, pp. 63–77.

[26] M. U. Chaudhry and J.-H. Lee, "Motifs: Monte carlo tree search based feature selection," *Entropy*, vol. 20, no. 5, 2018, Art. no. 385.

[27] M. U. Chaudhry and J.-H. Lee, "Feature selection for high dimensional data using Monte Carlo tree search," *IEEE Access*, vol. 6, pp. 76036–76048, 2018.

[28] G. Hinton et al., "Distilling the knowledge in a neural network," 2015, arXiv: *1503.02531*.

[29] S. Tan, R. Caruana, G. Hooker, and Y. Lou, "Distill-and-compare: Auditing black-box models using transparent model distillation," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 303–310.

[30] Z. Che, S. Purushotham, R. Khemani, and Y. Liu, "Distilling knowledge from deep networks with applications to healthcare domain," 2015, arXiv: *1512.03542*.

[31] N. Frosst and G. Hinton, "Distilling a neural network into a soft decision tree," 2017, arXiv: *1711.09784*.

[32] X. Liu, X. Wang, and S. Matwin, "Improving the interpretability of deep neural networks with knowledge distillation," in *Proc. IEEE Int. Conf. Data Mining Workshops*, 2018, pp. 905–912.

[33] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017.

[34] Z. Hu, X. Ma, Z. Liu, E. Hovy, and E. Xing, "Harnessing deep neural networks with logic rules," 2016, arXiv: *1603.06318*.

[35] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. Eur. Conf. Mach. Learn.*, Springer, 2006, pp. 282–293.

[36] M. S. N. S. J. Schaeffer et al., "Comparing UCT versus CFR in simultaneous games," in *Proc. IJCAI Workshop Gen. Game Playing*, 2009, pp. 75–82.

[37] M. Lanctot, C. Wittlinger, M. H. Winands, and N. G. Den Teuling, "Monte Carlo tree search for simultaneous move games: A case study in the game of tron," in *Proc. 25th Benelux Conf. Artif. Intell.*, 2013, pp. 104–111.

[38] P. D. Allison, *Multiple Regression: A Primer*. Newbury Park, CA, USA: Pine Forge Press, 1999.

[39] S. Kumar, F. Spezzano, and V. Subrahmanian, "VEWS: A wikipedia vandal early warning system," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2015, pp. 607–616.

[40] Y. Zhang et al., "RuDi: Explaining behavior sequence models by automatic statistics generation and rule distillation," in *Proc. Conf. Inf. Knowl. Manage.*, 2022, pp. 2651–2660.

[41] B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani, "Least angle regression," *Ann. Statist.*, vol. 32, no. 2, pp. 407–499, 2004.

[42] G. J. Székely, M. L. Rizzo, and N. K. Bakirov, "Measuring and testing dependence by correlation of distances," *Ann. Statist.*, vol. 35, no. 6, pp. 2769–2794, 2007.

[43] A. Chaudhuri and W. Hu, "A fast algorithm for computing distance correlation," *Comput. Statist. Data Anal.*, vol. 135, pp. 15–24, 2019.

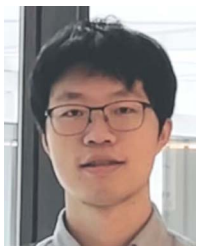
[44] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Evanston, IL, USA: Routledge, 2017.

[45] C. Molnar, Interpretable machine learning, 2022. [Online]. Available: https://christophm.github.io/interpretable-ml-book

Yao Zhang is currently working as a postdoctoral researcher with Fudan University, Shanghai, China. His research interests include graph mining and machine learning.

Yun Xiong received the PhD degree in computer and software theory from Fudan University. She is a professor of computer science with Fudan University, Shanghai, China. Her research interests include database and data mining.

Yiheng Sun received the BSc degree in information systems from Fudan University, China, in 2015. He is currently a senior algorithm engineer with Tencent, Shenzhen, China. He was a visiting scholar with Northwestern University, Illinois, USA, in 2016. His research interests include credit risk management and financial technology.



Yucheng Jin is currently working toward the doctor's degree with Fudan University, Shanghai, China. His research interests include graph mining and machine learning.

Caihua Shan received the PhD degree in computer science from the University of Hong Kong, in 2020. She is currently working with Microsoft Research Asia, focusing on graph data analysis and responsible AI.

Tian Lu received the PhD degree in information systems from Fudan University. He was a postdoc researcher with Carnegie Mellon University. He is an assistant professor of information systems with Arizona State University. His research interests include the impact of IT and Big Data, fintech, and human-AI interfaces.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.

Hui Song received the PhD degree in communication and information system from Wuhan University. He is currently working as a senior researcher with Tencent Weixin Group. His research interests include data mining and machine learning.

Shengli Sun received the PhD degree from Fudan University, Shanghai, China, in 2008. He is currently an associate professor with the School of Software and Microelectronics, Peking University, Beijing, China. His research interests include Big Data, data science, machine learning and their applications in intelligent risk control.

Authorized licensed use limited to: Mepco Schlenk Engineering College. Downloaded on December 12,2025 at 03:42:49 UTC from IEEE Xplore. Restrictions apply.